

Homework 4: Transformer

CSCI-GA 2572 Deep Learning
Fall 2025

The goal of homework 4 is to test your understanding of attention and Transformers.

In part 1, you should submit all your answers in a pdf file. As before, we recommend using L^AT_EX.

For part 2, you will implement some neural networks by adding your code to the provided `ipynb` file.

The due date of homework 4 is **11:55pm, 11/2**. Submit the following files in a zip file `your_net_id.zip` through NYU Classes:

- `hw4_theory.pdf`
- `hw4_impl.ipynb`

The following behaviors will result in penalty of your final score:

1. 10% penalty for submitting your file without using the correct naming format (including naming the zip file, PDF file or python file wrong, adding extra files in the zip folder, like the testing scripts in your zip file).
2. 10% penalty for every extra day of lateness. Up to 4 days max (after that we won't accept submission).
3. 20% penalty for code submission that cannot be executed following the steps we mentioned.

1 Theory (50pt)

1.1 Attention (13pts)

This question tests your intuitive understanding of attention and its property.

- (a) (1pts) Given queries $Q \in \mathbb{R}^{d \times n}$, keys $K \in \mathbb{R}^{d \times m}$ and values $V \in \mathbb{R}^{t \times m}$, describe the operations needed to calculate the output H of the standard dot-product attention. What is the output dimension? (You can use the softargmax_β function directly. It is applied to the column of each matrix).

$$S = K^\top Q \in \mathbb{R}^{m \times n}.$$

Apply a column-wise temperature softmax (a.k.a. softargmax_β) to get attention weights

$$A = \text{softargmax}_\beta(S) \in \mathbb{R}^{m \times n}.$$

The output is the value aggregation

$$H = VA \in \mathbb{R}^{t \times n}.$$

- (b) (2pts) Explain how the scale β influences the output of the attention? And what β is convenient to use?

$$\beta \rightarrow 0 : A_{.j} \rightarrow \frac{1}{m} \mathbf{1} \quad (\text{uniform averaging}), \quad \beta \rightarrow \infty : A_{.j} \rightarrow \text{one-hot}(\arg \max_i S_{ij}) \quad (\text{hard selection}).$$

Larger β makes attention more *selective*; smaller β makes it more *diffuse*. A convenient, widely used choice is

$$\beta = \frac{1}{\sqrt{d}},$$

which keeps the variance of dot-products roughly $\mathcal{O}(1)$ and stabilizes optimization.

- (c) (2pts) One advantage of the attention operation is that it is really easy to preserve a value vector v to the output h . Explain in what situation, the outputs preserves the value vectors. Also, what should the scale β be if we just want the attention operation to preserve value vectors. Which of the four types of attention we are referring to? How can this be done when using fully connected architectures?

When each query q_j is most similar to a single key k_i , the attention weights become nearly one-hot, so the output $h_j = v_i$. This happens when $\beta \rightarrow \infty$, making the softmax sharp. This corresponds to **hard attention**. In a fully connected architecture, it can be approximated by using large linear weights before the softmax, effectively implementing a near-argmax selection.

- (d) (2pts) On the other hand, the attention operation can also dilute different value vectors v to generate new output h . Explain in what situation the outputs is spread version of the value vectors. Also, what should the scale β be if we just want the attention operation to diffuse as much as possible. Which of the four types of attention we are referring to? How can this be done when using fully connected architectures?

When all queries attend equally to all keys, the output becomes a weighted average of all v_i , effectively diffusing the information. This occurs when $\beta \rightarrow 0$, flattening the softmax distribution. This corresponds to **soft attention**. In fully connected networks, this can be achieved by using small weight magnitudes before the softmax so that logits are close to uniform.

- (e) (2pts) If we have a small perturbation to one of the k_i (you could assume the perturbation is a zero-mean Gaussian with small variance, so the new $\hat{k}_i = k_i + \epsilon$), how will the output of H change?

A small perturbation to one key k_i slightly alters its similarity with all queries q_j , thus slightly changing the corresponding column of the attention matrix A . The resulting output $H = VA$ changes smoothly in proportion to the local gradient of the softmax, indicating that attention is locally stable to small key perturbations.

- (f) (2pts) If we have a small perturbation to one of the queries q_i , how will the output of H change? How would this differ from the previous case?

A small perturbation to one query q_i changes all similarities between q_i and every key k_j , thereby affecting the entire attention weight vector $A_{.i}$ and its corresponding output h_i . Unlike key perturbations, which affect multiple outputs slightly, query perturbations influence a single output more significantly.

- (g) (2pts) If we have a large perturbation that scales one key so that $\hat{k} = \alpha k$ for $\alpha > 1$, how will the output of H change?

If one key is scaled as $\hat{k} = \alpha k$ with $\alpha > 1$, its dot products with compatible queries increase, so its attention weights dominate. The corresponding value v will receive disproportionately higher attention, effectively amplifying its contribution to H and making the model more focused on that component.

1.2 Multi-headed Attention (3pts)

This question tests your intuitive understanding of Multi-headed Attention and its property.

- (a) (1pts) Given queries $Q \in \mathbb{R}^{d \times n}$, $K \in \mathbb{R}^{d \times m}$ and $V \in \mathbb{R}^{t \times m}$, describe the operations for calculating the output H of the standard multi-headed scaled dot-product attention. Assume we have h heads.

Multi-headed scaled dot-product attention first projects the inputs into h subspaces:

$$Q_i = W_i^Q Q, \quad K_i = W_i^K K, \quad V_i = W_i^V V, \quad \text{for } i = 1, \dots, h,$$

where W_i^Q, W_i^K, W_i^V are learnable projection matrices. Each head computes attention independently:

$$H_i = V_i \text{softmax}\left(\frac{K_i^\top Q_i}{\sqrt{d_h}}\right),$$

with $d_h = d/h$, ensuring $H_i \in \mathbb{R}^{t_h \times n}$. The outputs from all heads are concatenated and linearly projected:

$$H = W^O[H_1; H_2; \dots; H_h],$$

where W^O combines information from all heads into the final representation.

- (b) (2pts) Is there anything similar to multi-headed attention for convolutional networks? Explain why do you think they are similar.

Multi-headed attention is analogous to having multiple convolutional filters in CNNs. Each attention head, like a convolutional filter, learns to focus on a different aspect or pattern of the input. While CNN filters extract spatially localized features, attention heads capture relationships across positions or tokens. Both enable the model to represent diverse information in parallel through separate learned channels.

1.3 Self Attention (11pts)

This question tests your intuitive understanding of Self Attention and its property.

- (a) (2pts) Given an input $C \in \mathbb{R}^{e \times n}$, what are the queries Q , the keys K and the values V and the output H of the standard multi-headed scaled dot-product self-attention? Assume we have h heads. (You can name and define the weight matrices by yourself.)

In self-attention, the queries, keys, and values are all derived from the same input $C \in \mathbb{R}^{e \times n}$. For each head $i \in \{1, \dots, h\}$:

$$Q_i = W_i^Q C, \quad K_i = W_i^K C, \quad V_i = W_i^V C,$$

where $W_i^Q, W_i^K \in \mathbb{R}^{d_h \times e}$ and $W_i^V \in \mathbb{R}^{t_h \times e}$ are learnable weight matrices. Each head computes:

$$H_i = V_i \text{softmax}\left(\frac{K_i^\top Q_i}{\sqrt{d_h}}\right),$$

so that $H_i \in \mathbb{R}^{t_h \times n}$. The final output is obtained by concatenation and projection:

$$H = W^O[H_1; H_2; \dots; H_h],$$

with $W^O \in \mathbb{R}^{e \times (ht_h)}$.

- (b) (2pts) Explain what is positional encoding. What is the difference between absolute and relative positional encoding. When is it appropriate to use absolute positional encoding? When is it more appropriate to use relative encoding?

Positional encoding injects information about token order into the input sequence since self-attention alone is permutation-invariant. **Absolute positional encoding** assigns each position a fixed embedding (e.g., sinusoidal or learned) added to the input. **Relative positional encoding** represents the distance or offset between tokens, enabling position-invariant relationships. Absolute encodings are more suitable for fixed-length sequences (e.g., text classification), while relative encodings work better for variable-length or streaming tasks (e.g., language modeling, speech).

- (c) (2pts) Show one situation where the self-attention layer behaves like an identity layer or permutation layer.

Self-attention behaves like an identity or permutation layer when the attention matrix is close to the identity or a permutation matrix. For instance, if $Q = K$ and $\beta \rightarrow \infty$, the softmax becomes one-hot on the diagonal (or permuted diagonal), yielding $H = V$, which reproduces or reorders the input without mixing information across tokens.

- (d) (3pts) Show one situation where the self-attention layer behaves like a convolution layer with a kernel larger than 1. You can assume we use positional encoding.

Self-attention acts like a convolution with a kernel larger than 1 when each token attends primarily to its nearby neighbors. With positional encodings encoding distance, and attention weights localized around adjacent positions, the output h_i becomes a weighted average of neighboring embeddings—analogous to applying a wide convolutional filter that aggregates context over multiple positions.

- (e) (2pts) Suppose we are training a transformer architecture for real-time automatic speech recognition. Do we need to do anything special to the attention mechanism? How do we achieve this?

For real-time speech recognition, the model must operate causally—each output should depend only on past and current inputs. To achieve this, we use **causal (masked) self-attention**, where future positions are masked out in the softmax computation so that attention weights satisfy $A_{ij} = 0$ for $j > i$. This ensures streaming and online inference without future information leakage.

1.4 Transformer (15pts)

Read the original paper on the Transformer model: “*Attention is All You Need*” by Vaswani et al. (2017).

- (a) (3pts) Explain the primary differences between the Transformer architecture and previous sequence-to-sequence models (such as RNNs and LSTMs).

The Transformer replaces recurrence and convolution with attention mechanisms that model dependencies between all tokens in parallel. Unlike RNNs or LSTMs, which process sequences sequentially and thus suffer from long-term dependency and inefficiency issues, the Transformer uses self-attention to capture global context in a single step. This allows full parallelization during training, reduces path length between tokens, and leads to more efficient and scalable sequence modeling.

- (b) (3pts) Explain the concept of self-attention and its importance in the Transformer model.

Self-attention allows each token to dynamically attend to other tokens in the same sequence to compute a contextualized representation. It enables the model to understand relationships—such as syntactic or semantic dependencies—regardless of their distance in the sequence. In Transformers, self-attention forms the core of both encoder and decoder layers, providing a flexible and globally-aware representation without recurrence.

- (c) (3pts) Describe the multi-head attention mechanism and its benefits.

Multi-head attention projects queries, keys, and values into multiple subspaces and performs attention in each independently:

$$\text{head}_i = \text{Attn}(QW_i^Q, KW_i^K, VW_i^V).$$

The outputs from all heads are concatenated and projected to form the final result. This allows the model to capture different types of relationships (e.g., syntactic vs. semantic) in parallel and improves expressiveness by attending to multiple representation subspaces simultaneously.

- (d) (3pts) Explain the feed-forward neural networks used in the model and their purpose.

Each Transformer layer includes a position-wise feed-forward network (FFN) applied independently to each token:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

The FFN increases the model’s capacity by performing non-linear transformations on each token’s representation after attention. It acts as a per-token MLP that refines information extracted by the attention mechanism.

- (e) (3pts) Name two techniques used in the paper to improve training stability of the transformer model, in particular with regard to the issue of exploding / vanishing gradients. Briefly explain how they do so.

Two techniques that stabilize Transformer training are:

- **Residual connections and layer normalization:** Residual (skip) connections allow gradients to flow directly across layers, mitigating vanishing/exploding gradients, while layer normalization standardizes activations to maintain stable distributions.

- **Scaled dot-product attention:** Dividing by $\sqrt{d_k}$ in the attention logits prevents large dot products that could cause softmax saturation and unstable gradients, ensuring smoother optimization.

1.5 Vision Transformer (8pts)

Read the paper on the Transformer model: “*An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale*”.

- (a) (2pts) What is the key difference between the Vision Transformer (ViT) and traditional convolutional neural networks (CNNs) in terms of handling input images? Can you spot a convolution layer in the ViT architecture?

The Vision Transformer (ViT) divides an image into fixed-size patches (e.g., 16×16), flattens each patch, and linearly projects it into an embedding vector—treating patches as tokens in a sequence. In contrast, CNNs use convolutional kernels to process local pixel neighborhoods hierarchically. ViT does not rely on convolutional layers; instead, it uses a linear projection layer to embed image patches, which serves a similar purpose as an initial convolution but without spatial weight sharing.

- (b) (2pts) What is the role of positional embeddings in the Vision Transformer model, and how do they differ from positional encodings used in the original Transformer architecture?

Positional embeddings in ViT inject information about patch order since the transformer architecture is permutation-invariant. They are learnable vectors added to patch embeddings, allowing the model to infer spatial relationships between patches. Unlike the fixed sinusoidal positional encodings used in the original Transformer, ViT learns positional embeddings directly from data, enabling more flexible adaptation to visual structures.

- (c) (2pts) How does the Vision Transformer model generate the final classification output? Describe the process and components involved in this step.

ViT prepends a special learnable [CLS] (classification) token to the sequence of patch embeddings. After several layers of multi-head self-attention and feed-forward blocks, the final hidden state corresponding to the [CLS] token serves as a global image representation. This representation is then passed through a linear classification head (usually a single fully connected layer) to produce the final class logits.

- (d) (2pts) How does ViT compare with CNN in terms of performance across different data regimes? What explains this trend?

ViT matches or surpasses CNN performance only when trained on large-scale datasets (e.g., JFT-300M or ImageNet-21k). In smaller data regimes, CNNs perform better due to their strong inductive biases such as locality and translation equivariance, which help generalization with limited data. ViTs, having weaker built-in biases, require large datasets or pretraining to learn these relationships effectively.

2 Implementation (50pt)

Please add your solutions to this notebook `HW4-VIT-Student.ipynb`. Please use your NYU account to access the notebook. The notebook contains parts marked as **TODO**, where you should put

your code or explanations. The notebook is a Google Colab notebook, you should copy it to your drive, add your solutions, and then download and submit it to NYU Classes. You're also free to run it on any other machine, as long as the version you send us can be run on Google Colab.